



**UNIVERSIDADE FEDERAL DE SERGIPE
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA
DEPARTAMENTO DE COMPUTAÇÃO**

**Sistemas Operacionais - 2026.2
Trilha A — Chat local com Ollama + Open WebUI**

Enzo Emanuel Costa Maia
Filipe de Carvalho Godoy
Giulian Fábio Bastos Amorim Lima
José Fagner Silva Junqueira
Katyane dos Santos
Rauany Ingrid Santos de Jesus
Vênisson Cardoso dos Santos

**São Cristóvão, Sergipe
2026**

****Nota:**** Todos os códigos-fonte, prompts e demais artefatos de análise utilizados para a elaboração deste tutorial estão disponíveis publicamente em nosso repositório no GitHub: > https://github.com/Enz0EMCosta/SO_2026_2_TrilhaA

****Nota:**** Link para a pasta com vídeo do grupo:
https://drive.google.com/file/d/1qio1HxR_OiQHEN639TeSWmgItDoIr36o/view?usp=drive_link

- **1 - Enzo Emanuel Maia Costa (202300061901)**

Modelo analisado: *google/gemma-4-E2B-it-qat-q4_0-gguf*

Tarefa: Preparar e documentar o ambiente experimental utilizado pela equipe para a execução local do modelo de linguagem, realizando o inventário dos recursos computacionais, a instalação e configuração do Ollama, o download e a execução inicial do modelo, além da preparação do Open WebUI como interface de interação. Também faz parte da tarefa registrar as características do ambiente e identificar suas limitações de hardware, especialmente em relação à CPU, memória RAM, armazenamento e disponibilidade de GPU.

Análise da tarefa: A atividade foi desenvolvida considerando duas configurações de ambiente utilizadas pelos integrantes da equipe. Parte do grupo realizou a preparação utilizando **WSL2**, com Ubuntu 24.04.1 LTS, enquanto outra parte utilizou uma **máquina virtual através do VirtualBox**, também com ambiente Linux. A utilização dessas duas abordagens permitiu executar o mesmo modelo e runtime em ambientes distintos, possibilitando observar como diferentes formas de disponibilização do ambiente e de alocação de recursos podem influenciar a execução local de modelos de linguagem. A atividade foi dividida nas seguintes etapas:

- **Inventário do ambiente:** identificação do sistema operacional, kernel, arquitetura, CPU, quantidade de núcleos e threads, memória RAM, swap, armazenamento e disponibilidade de GPU;
- **Instalação e execução:** preparação do sistema, instalação do Ollama, inicialização do servidor, disponibilização do modelo *google/gemma-4-E2B-it-qat-q4_0-gguf*, execução de uma inferência inicial e preparação do Open WebUI.

A análise do ambiente é importante porque a execução local de modelos depende diretamente dos recursos computacionais disponíveis, a quantidade de RAM, por exemplo, influencia a capacidade de carregar o modelo e manter o contexto da interação, enquanto a disponibilidade de GPU pode alterar significativamente o desempenho da inferência.

No ambiente WSL2 utilizado como referência para o inventário, foram identificados **4 núcleos físicos e 8 threads**, utilizando um processador **Intel Core i5-8265U**, aproximadamente **6,1 GiB de memória RAM** e **2 GiB de memória swap**. Durante a inicialização do Ollama, o runtime identificou a execução utilizando **CPU**, com **0 B de VRAM disponível para o processo de inferência**.

Guia de execução e evidências:

Para a identificação do sistema operacional utilizado no ambiente experimental, foram utilizados os comandos `cat /etc/os-release` e `uname -a`. O primeiro apresenta informações sobre a distribuição Linux instalada, enquanto o segundo permite identificar o kernel, a arquitetura e outras características do sistema.

```
PS C:\Users\Enzom> wsl
enzo@DESKTOP-BHUFF6Q:/mnt/c/Users/Enzom$ cat /etc/os-release
PRETTY_NAME="Ubuntu 24.04.1 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.1 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=noble
LOGO=ubuntu-logo
enzo@DESKTOP-BHUFF6Q:/mnt/c/Users/Enzom$ uname -a
Linux DESKTOP-BHUFF6Q 5.10.16.3-microsoft-standard-WSL2 #1 SMP Fri Apr 2 22:23:49 UTC 2021 x86_64 x86_64 x86_64 GNU/Linux
enzo@DESKTOP-BHUFF6Q:/mnt/c/Users/Enzom$
```

Para identificar as características do processador disponível no ambiente, foi utilizado o comando `lscpu`, que apresenta informações como modelo da CPU, quantidade de núcleos, quantidade de threads, arquitetura e frequência, e para a quantidade de processadores lógicos disponíveis para o ambiente, foi utilizado o comando `nproc`.

```

enzo@DESKTOP-BHUFF6Q: /mnt/c/Users/Enzon$ uname -a
Linux DESKTOP-BHUFF6Q 5.10.16.3-microsoft-standard-WSL2 #1 SMP Fri Apr 2 22:23:49 UTC 2021 x86_64 x86_64 x86_64 GNU/Linux
enzo@DESKTOP-BHUFF6Q: /mnt/c/Users/Enzon$ lscpu
Architecture:            x86_64
CPU op-mode(s):          32-bit, 64-bit
Address sizes:            39 bits physical, 48 bits virtual
Byte Order:               Little Endian
CPU(s):                   8
On-line CPU(s) list:      0-7
Vendor ID:                GenuineIntel
Model name:               Intel(R) Core(TM) i5-8265U CPU @ 1.60GHz
CPU family:               6
Model:                    142
Thread(s) per core:       2
Core(s) per socket:       4
Socket(s):                1
Stepping:                 12
BogoMIPS:                 3600.00
Flags:                    fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ss ht syscall nx pdpe1gb rdtscp lm const
                        ant_tsc rep_good nopl xtopology cpuid pni pclmulqdq ssse3 fma cx16 pcid sse4_1 sse4_2 movbe popcnt aes xsave avx f16c rdrand hypervisor la
                        hf_lm abm 3dnowprefetch invpcid_single ssbd ibrs ibpb stibp ibrs_enhanced fsgsbase bmi1 avx2 smep bmi2 erms invpcid rdseed adx smap clflush
                        hopt xsaveopt xsavec xgetbv1 xsaves md_clear flush_l1d arch_capabilities
Virtualization features:
Hypervisor vendor:        Microsoft
Virtualization type:      full
Caches (sum of all):
L1d:                      128 KiB (4 instances)
L1i:                      128 KiB (4 instances)
L2:                        1 MiB (4 instances)
L3:                        6 MiB (1 instance)
Vulnerabilities:
Itlb multihit:             KVM: Mitigation: VMX unsupported
L1tf:                     Not affected
Mds:                      Not affected
Meltdown:                 Not affected
Spec store bypass:         Mitigation; Speculative Store Bypass disabled via prctl and seccomp
Spectre v1:               Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2:               Mitigation; Enhanced IBRS, IBPB conditional, RSB filling
Srbds:                    Mitigation; TSX disabled
Tsx async abort:          Not affected
enzo@DESKTOP-BHUFF6Q: /mnt/c/Users/Enzon$ nproc
8
enzo@DESKTOP-BHUFF6Q: /mnt/c/Users/Enzon$

```

Para verificar a quantidade de memória RAM disponível, o consumo atual e a quantidade de memória swap configurada no ambiente, foi utilizado o comando `free -h`. A opção `-h` permite apresentar os valores em unidades de leitura simplificada, como MiB e GiB. Para verificar a capacidade, o espaço utilizado e o espaço disponível nos sistemas de arquivos, foi utilizado o comando `df -h`.

```

enzo@DESKTOP-BHUFF6Q: /mnt/c/Users/Enzon$ nproc
8
enzo@DESKTOP-BHUFF6Q: /mnt/c/Users/Enzon$ free -h
Mem: 6.1Gi 234Mi 5.9Gi 68Ki 62Mi 5.9Gi
Swap: 2.0Gi 0B 2.0Gi
enzo@DESKTOP-BHUFF6Q: /mnt/c/Users/Enzon$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sdb        251G  4.3G  234G   2% /
tmpfs           3.1G   0  3.1G   0% /mnt/wsl
tools          222G  176G   46G  80% /init
none           3.1G  4.0K   3.1G   1% /run
none           3.1G   0   3.1G   0% /run/lock
none           3.1G   0   3.1G   0% /run/shm
none           3.1G   0   3.1G   0% /run/user
tmpfs          3.1G   0   3.1G   0% /sys/fs/cgroup
drivers         222G  176G   46G  80% /usr/lib/wsl/drivers
lib            222G  176G   46G  80% /usr/lib/wsl/lib
C:\            222G  176G   46G  80% /mnt/c
enzo@DESKTOP-BHUFF6Q: /mnt/c/Users/Enzon$

```

Para complementar a análise do armazenamento e identificar os discos reconhecidos pelo sistema, foi utilizado o comando `lsblk`.

```

C:\            222G  176G   46G  80% /mnt/c
enzo@DESKTOP-BHUFF6Q: /mnt/c/Users/Enzon$ lsblk
NAME
MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda  8:0   0  256G  0 disk
sdb  8:16  0  256G  0 disk /
enzo@DESKTOP-BHUFF6Q: /mnt/c/Users/Enzon$

```

Para verificar a disponibilidade de uma GPU NVIDIA no ambiente experimental, foi utilizado o comando `nvidia-smi`. Quando disponível, este comando apresenta informações sobre a GPU, incluindo modelo, memória de vídeo e utilização dos recursos. Após a preparação do ambiente, foi realizada a instalação do Ollama, e para verificar se a instalação foi concluída corretamente e identificar a versão utilizada:

```
enzo@DESKTOP-BHUFF6Q:/mnt/c/Users/EnzoM$ lsblk
NAME
MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda  8:0    0  256G  0 disk
sdb  8:16   0  256G  0 disk /
enzo@DESKTOP-BHUFF6Q:/mnt/c/Users/EnzoM$ nvidia-smi
Command 'nvidia-smi' not found, but can be installed with:
sudo apt install nvidia-utils-525          # version 525.147.05-0ubuntu1, or
sudo apt install nvidia-utils-525-server   # version 525.147.05-0ubuntu1
sudo apt install nvidia-utils-470          # version 470.256.02-0ubuntu0.24.04.1
sudo apt install nvidia-utils-470-server   # version 470.256.02-0ubuntu0.24.04.1
sudo apt install nvidia-utils-535          # version 535.309.01-0ubuntu0.24.04.1
sudo apt install nvidia-utils-535-server   # version 535.309.01-0ubuntu0.24.04.1
sudo apt install nvidia-utils-580          # version 580.159.03-0ubuntu0.24.04.1
sudo apt install nvidia-utils-580-server   # version 580.159.03-0ubuntu0.24.04.1
sudo apt install nvidia-utils-565-server   # version 565.57.01-0ubuntu0.24.04.3
sudo apt install nvidia-utils-550-server   # version 550.163.01-0ubuntu0.24.04.1
sudo apt install nvidia-utils-595          # version 595.71.05-0ubuntu0.24.04.1
sudo apt install nvidia-utils-595-server   # version 595.71.05-0ubuntu0.24.04.1
enzo@DESKTOP-BHUFF6Q:/mnt/c/Users/EnzoM$ ollama --version
ollama: could not connect to a running Ollama instance
```

No ambiente utilizado, o Ollama identificou a execução por CPU e registrou **0 B de VRAM**, caracterizando a utilização da CPU como recurso de processamento do modelo nesse ambiente.

Para iniciar o servidor responsável pelo gerenciamento e execução dos modelos locais, foi utilizado o comando `ollama serve`, onde esse processo disponibiliza a API local do Ollama para que os comandos e aplicações possam se comunicar com o runtime.

```
Experimente a nova plataforma cruzada PowerShell https://aka.ms/pscore6
PS C:\Users\EnzoM> wsl
enzo@DESKTOP-BHUFF6Q:/mnt/c/Users/EnzoM$ ollama --version
ollama version is 0.34.0
enzo@DESKTOP-BHUFF6Q:/mnt/c/Users/EnzoM$ ollama list
NAME      ID      SIZE    MODIFIED
enzo@DESKTOP-BHUFF6Q:/mnt/c/Users/EnzoM$ ollama serve
Error: listen tcp 127.0.0.1:11434: bind: address already in use
enzo@DESKTOP-BHUFF6Q:/mnt/c/Users/EnzoM$
```

Após a disponibilização do modelo, foi realizada uma execução inicial através do terminal. O objetivo dessa etapa é verificar se o Ollama consegue carregar o modelo e realizar uma inferência local. Como exemplo, fiz a clássica pergunta “O que é um Sistema operacional, resuma em uma frase?”, e ele retornou corretamente seu significado.

```

benzo@DESKTOP-BHUFF6Q: ~/nt/c/Users/Benzo$ ollama run hf.co/google/gemma-4-E2B-1t-qat-q4_0-gguF:latest "Responda em português: o que é um sistema operacional? Use apenas uma frase."
<|channel>thought
Thinking Process:
1. **Analyze the Request:** The user wants to know "o que é um sistema operacional?" (what is an operating system?) and specifically asked for the answer in Portuguese. The crucial constraint is: "Use apenas uma frase" (Use only one sentence).
2. **Identify Key Concepts:**
  * System Software (Software fundamental).
  * Manages hardware resources (Gerencia recursos de hardware).
  * Acts as an interface between hardware and applications (Interface entre hardware e aplicações).
  * Core function (Orquestrar e controlar).
3. **Draft Potential Sentences (Focusing on conciseness and comprehensiveness):**
  * Option 1 (Basic): Um sistema operacional é um software que gerencia os recursos do computador e permite a execução de outros programas.
  * Option 2 (More detailed): É o software fundamental que atua como intermediário entre o hardware do computador e os softwares aplicativos, controlando e coordenando todas as operações.
4. **Select the Best Single Sentence:** Option 1 is clear, direct, and fits the "one sentence" constraint perfectly while covering the main function.
5. **Final Review:**
  * Language: Portuguese (Check).
  * Content: Defines OS (Check).
  * Length: One sentence (Check).
6. **Output Generation:** (Portuguese response)<|channel>Um sistema operacional é o software fundamental que gerencia os recursos de hardware e fornece a plataforma necessária para a execução de outros programas e aplicações.
benzo@DESKTOP-BHUFF6Q: ~/nt/c/Users/Benzo$

```

Resultados e Conclusão

A partir do inventário e da preparação do ambiente, foi possível estabelecer uma configuração experimental capaz de executar o runtime Ollama e utilizar o modelo definido pela equipe para os experimentos. A utilização de WSL2 e VirtualBox permitiu trabalhar com duas estratégias de virtualização, mantendo como elementos comuns o **Ubuntu/Linux**, **Ollama** e o **modelo Gemma**, tal organização forneceu uma base para os experimentos posteriores relacionados ao consumo de recursos, tamanho do contexto, quantização e desempenho da inferência, onde além da execução pelo terminal, o **Open WebUI** foi definido como camada de interface gráfica, permitindo que o modelo seja utilizado por meio de uma interface de conversação enquanto o Ollama permanece responsável pelo processamento local. Também no ambiente WSL2 analisado, foi possível identificar algumas limitações importantes. A execução do modelo ocorreu utilizando a CPU, sem VRAM disponível para o runtime, enquanto a memória disponível para o ambiente era de aproximadamente 6,1 GiB. Além disso, o modelo instalado ocupa aproximadamente 4,3 GB de armazenamento, evidenciando que a capacidade de armazenamento e, principalmente, a memória disponível devem ser consideradas na execução de modelos locais.

Logo, concluímos que a tarefa permitiu estruturar o ambiente necessário para a execução local do modelo definido pela equipe, abrangendo desde a identificação dos recursos computacionais até a instalação do runtime e disponibilização do modelo para inferência, e a divisão entre **WSL2** e **máquina virtual** permitiu que diferentes integrantes utilizassem configurações distintas sem alterar o objetivo principal do experimento, em que o inventário realizado também possibilitou identificar as limitações de hardware que deverão ser consideradas nas etapas posteriores da atividade. Dessa forma, a preparação do ambiente fornece a infraestrutura necessária para as próximas análises da equipe, especialmente aquelas relacionadas ao consumo de memória, impacto do tamanho do contexto, quantização, armazenamento e desempenho da execução local do modelo.

2 - Open WebUI na Trilha A

Giulian Fabio Bastos Amorim Lima

A interface foi instalada e produziu respostas com o modelo escolhido. Os testes também revelaram limites de memória e contexto. Este registro resume a seção 6.2, a Q1 e a contribuição à Q12, com evidências na página 3.

Ficha da aplicação

Item	Registro
Nome e finalidade	Open WebUI — interface web para selecionar modelos, conversar e manter histórico local.
Autoria e repositório	Timothy Jaeryang Baek; organização open-webui e colaboradores. https://github.com/open-webui/open-webui [1]
Licença	Open WebUI License, com condições de marca e contribuições sob licenças anteriores. A identidade visual foi preservada. [2]
Versão e acesso	v0.11.3 nos logs; imagem ghcr.io/open-webui/open-webui:main. Fontes acessadas em 10/09/2026. Commit Git não coletado.
Ambiente	Ubuntu 24.04.4 LTS no Oracle VirtualBox; 2 CPUs virtuais; última configuração: 3,8 GiB de RAM e 4 GiB de swap.
Dependências	Docker 29.1.3; Ollama 0.34.0. Aplicação com Python, FastAPI/Uvicorn, SvelteKit e SQLite. Versões internas não coletadas. [1]
Persistência	Volume open-webui em /app/backend/data, para banco SQLite, histórico e cache. Pesos do Ollama armazenados no Ubuntu.

Arquitetura utilizada

O navegador no Windows acessa 127.0.0.1:3001. Um túnel SSH encaminha esse acesso ao Open WebUI, na porta 8081 do Ubuntu. O contêiner usa --network host e chama a API do Ollama em 127.0.0.1:11434. O Ollama executa o modelo na CPU e devolve a resposta à interface. Portanto, o loopback é o do Ubuntu; host.docker.internal não foi usado.

Q1. Por que essa trilha é adequada

A Trilha A permite estudar uma aplicação real com contêiner, serviço HTTP e processo de inferência. Observamos portas, persistência, uso de RAM e swap e encerramento por falta de memória. Esse ambiente também permite os experimentos da equipe sobre processos, threads, escalonamento e concorrência, relacionando a teoria de Sistemas Operacionais ao comportamento da aplicação.

Q2. Como a Open Web conversa com Ollama via host.docker.internal

O Open WebUI foi executado em um contêiner Docker dentro da máquina virtual Ubuntu, enquanto o Ollama foi instalado diretamente no Ubuntu. O Docker fornece o ambiente da aplicação e mantém seus dados em um volume persistente. Utilizamos --network host para compartilhar a rede do Ubuntu com o contêiner e configuramos OLLAMA_BASE_URL=http://127.0.0.1:11434. Dessa forma, o Open WebUI envia as mensagens à API do Ollama, que executa o Gemma e devolve as respostas para exibição no navegador.

O navegador envia a mensagem ao Open WebUI, pelo acesso em 127.0.0.1:3001, encaminhado pelo túnel SSH à porta 8081 do Ubuntu.

O Open WebUI envia uma requisição HTTP ao Ollama, em 127.0.0.1:11434, com o modelo escolhido e as mensagens da conversa.

O Ollama executa o Gemma na CPU e devolve o texto gerado ao Open WebUI.

O Open WebUI apresenta a resposta no navegador.

Instalação e resultados observados

Com o Ollama já instalado no Ubuntu, a aplicação foi executada com Docker. A configuração final incorpora a mudança da porta 8080 para 8081. [3, 4]

```
sudo apt install docker.io
sudo systemctl enable --now docker
sudo docker pull ghcr.io/open-webui/open-webui:main
sudo docker run -d --name open-webui --network host \
  -e PORT=8081 -e OLLAMA_BASE_URL=http://127.0.0.1:11434 \
  -v open-webui:/app/backend/data --restart unless-stopped \
  ghcr.io/open-webui/open-webui:main
```

Acesso pelo PowerShell do Windows, com a porta 2222 encaminhada à porta 22 da VM:

```
ssh -N -o ExitOnForwardFailure=yes -p 2222 `
  -L 3001:127.0.0.1:8081 vboxuser@127.0.0.1
```

Manter o túnel SSH aberto e acessar http://127.0.0.1:3001 no navegador.

Ocorrência	Adaptação e resultado
Falta de memória	O kernel encerrou llama-server por OOM. Foi ativada swap de 4 GiB. Após redução de serviços, a RAM disponível chegou a 3,2 GiB, antes de carregar o modelo.
Contexto excedido	Uma solicitação de 5.447 tokens ultrapassou o contexto de 4.096. Após ajustes na interface e novo teste curto, houve resposta (figura 1).
Geração demorada	Houve geração prolongada e forte uso de swap. O teste final com --think=false respondeu em uma frase no terminal (figura 2).

Limitações e validade dos resultados

O Windows possui 8 GB de RAM e chegou a 92% de uso (figura 3). A VM tem pouca memória e executa o modelo somente na CPU; a swap ajuda a evitar falhas, mas pode elevar a latência. As

respostas comprovam funcionamento básico, sem demonstrar estabilidade sob concorrência. Não houve medição controlada de desempenho. A inicialização também acessou o Hugging Face para baixar um modelo auxiliar; operação totalmente offline não foi verificada.

Pendências para o registro definitivo: versões internas das dependências, confirmação do ajuste de raciocínio no Open WebUI e testes controlados.

Evidências da instalação e dos testes

Capturas fornecidas por Giulian, em momentos diferentes da configuração. Elas documentam os resultados pontuais descritos nas legendas.

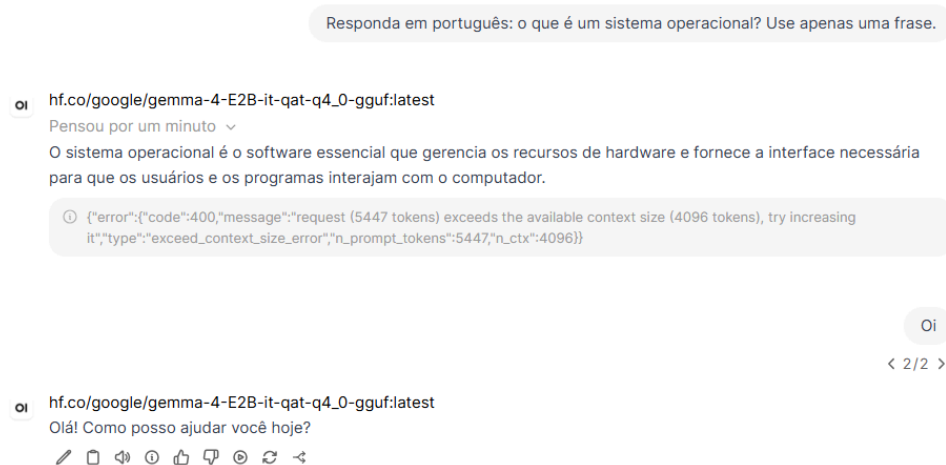


Figura 1 — Open WebUI com o Gemma selecionado e respostas em português. A captura também mantém o erro de contexto de uma tentativa anterior; o sucesso do chat curto não elimina essa limitação.

```
vboxuser@so2: $ free -h
              total        used        free      shared  buff/cache   available
Mem:           3.8Gi        663Mi        2.9Gi        344Ki        531Mi        3.2Gi
Swap:          4.0Gi         1.1Gi        2.9Gi

vboxuser@so2: $ sudo systemctl start ollama
vboxuser@so2: $ ollama run hf.co/google/gemma-4-E2B-it-qat-q4_0-gguf:latest --think=false "Responda em português, em uma frase: o que é um sistema operacional?"
Um sistema operacional é um software fundamental que gerencia os recursos do hardware de um computador e fornece uma plataforma para a execução de outros programas.
vboxuser@so2: $
```

Figura 2 — Teste no terminal com --think=false: resposta em uma frase. O free -h anterior à execução mostra 3,2 GiB disponíveis; não representa o consumo durante a inferência.

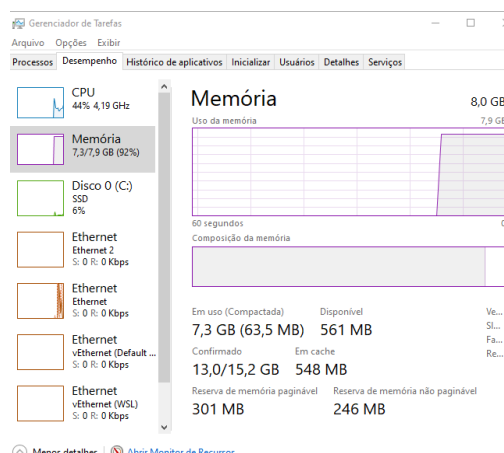


Figura 3 — Windows com 8 GB instalados, 92% de memória em uso e 561 MB disponíveis. Evidência de pressão de memória no computador que hospeda a VM, em uma etapa anterior.

• 3 - Ficha técnica — [google/gemma-4n-E2B-it](#)

Rauany Ingrid Santos de Jesus (202300061760)

Tarefa: Preencher a tabela de critérios do enunciado (identificação, parâmetros, finalidade, idioma, licença, formato, quantização, contexto, tamanho dos arquivos, requisitos de hardware, compatibilidade, documentação, riscos/limitações) para o [google/gemma-3n-E2B-it](#). Além de responder Q2 (modelo escolhido, parâmetros, por quê) e Q13 (o que precisou ser verificado sobre respostas de IA).

Critério	O que foi verificado
Identificação	Organização: Google (DeepMind). Repositório: google/gemma-4-E2B-it-qat-q4_0-gguf URL do model card: https://huggingface.co/google/gemma-4-E2B-it-qat-q4_0-gguf
Parâmetros	Arquitetura densa: 2,3B de parâmetros "efetivos" (5,1B contando as camadas de embedding — técnica PLE, Per-Layer Embeddings, mesma lógica de eficiência usada no Gemma 3n). 35 camadas, vocabulário de 262K tokens. Está dentro do limite de 10B exigido pela atividade.
Finalidade	Variante instruct/chat ("it") , com modo de "thinking" configurável (raciocínio passo a passo opcional), function calling nativo e suporte a papel de sistema (system) — adequada à Trilha A (chat).

Idioma	Pré-treinado com dados em mais de 140 idiomas; o card cita suporte "out-of-the-box" a 35+ idiomas para as capacidades principais. Não há confirmação explícita de qualidade em português — deve ser testado empiricamente pela equipe.
Licença	Apache 2.0 , conforme indicado na própria página deste repositório GGUF — diferente da licença "Gemma Terms of Use" usada na família Gemma 3n. Vale checar se essa mesma licença Apache 2.0 se aplica a todos os checkpoints da família Gemma 4 ou só a este derivado quantizado.
Formato	GGUF (formato "ready-to-deploy" para llama.cpp/Ollama/LM Studio), derivado de um checkpoint QAT em meia precisão.
Quantização	Q4_0 obtida via <i>Quantization-Aware Training</i> (QAT) — técnica que treina o modelo já considerando a quantização, preservando qualidade próxima do BF16 mas com memória bem menor que uma quantização pós-treino comum.
Contexto	Janela de contexto de 128 mil tokens (Gemma 4 E2B/E4B; os tamanhos maiores da família chegam a 256K).
Tamanho dos arquivos	O arquivo Q4_0 listado na página tem 3,35 GB . Ainda assim, é recomendável confirmar esse valor após o download real no ambiente da equipe (du -sh ou ollama list), pois pode variar por revisão do repositório.
Requisitos de hardware	Não há uma tabela oficial de RAM/VRAM mínima específica; pelo tamanho do arquivo (~3,35 GB em disco) e ao efetivo de ~2,3B parâmetros, é razoável estimar viabilidade em CPU com 8 GB de RAM livres.
Compatibilidade	Compatível com Ollama — mas por meio da integração direta com Hugging Face. Também compatível com llama.cpp, LM Studio, Jan, Unsloth Desktop, Docker Model Runner, entre outros.
Documentação	Model card bastante completo, incluindo relatório técnico próprio no arXiv, tabelas de benchmark comparando E2B/E4B/12B/26B/31B, boas práticas de uso (parâmetros de amostragem, modo de raciocínio, ordem de modalidades) e páginas complementares no Google AI e GitHub.

Riscos e limitações	O card repete o padrão de limitações da família Gemma: dependência da qualidade/diversidade dos dados de treino, dificuldade com tarefas abertas muito complexas, ambiguidade da linguagem natural, possibilidade de gerar afirmações factualmente incorretas (o modelo não é uma base de conhecimento) e limitações de senso comum. A avaliação de segurança indica melhorias importantes em relação às gerações anteriores (Gemma 3 e 3n), mas ainda assim recomenda salvaguardas de conteúdo por parte de quem implementa a aplicação.
----------------------------	---

Q2 — Qual modelo foi selecionado, quais são seus parâmetros e por que ele foi escolhido?

Foi selecionado o google/gemma-4-E2B-it-qat-q4_0-gguf, a versão quantizada em GGUF do modelo instruct Gemma 4 E2B do Google. Em termos de parâmetros, o modelo tem uma arquitetura densa com 2,3 bilhões de parâmetros "efetivos" (5,1 bilhões contando as camadas de embedding, via a técnica PLE — Per-Layer Embeddings), o que o mantém confortavelmente dentro do limite de 10B exigido pela atividade, e coloca esse modelo entre os menores da família Gemma 4 (que também inclui E4B, 12B, 26B A4B e 31B).

A escolha se justifica por:

- **Formato já pronto para execução local:** por já vir em GGUF quantizado (Q4_0, 3,35 GB), integra-se diretamente com o Ollama, atendendo ao runtime obrigatório da Trilha A sem etapas extras de conversão.
- **Qualidade preservada pela quantização QAT:** diferente de uma quantização pós-treino comum, o QAT treina o modelo já ciente da futura quantização, reduzindo a perda de qualidade em relação ao BF16 original — relevante para observar desempenho realista mesmo rodando em 4 bits.
- **Viabilidade de hardware:** o tamanho compacto do arquivo (3,35 GB) e o footprint efetivo de ~2,3B parâmetros tornam o modelo executável em hardware modesto (CPU + RAM limitada), compatível com o perfil de máquina da equipe.
- **Documentação e comparabilidade:** o model card é recente (relatório técnico de julho de 2026) e detalhado, com benchmarks próprios comparando o E2B a outras variantes da mesma família, o que facilita justificar a escolha do tamanho e discutir trade-offs no relatório.
- **Recursos de chat modernos:** suporte nativo a modo de raciocínio configurável ("thinking"), function calling e papel de sistema (system), úteis para explorar cenários de interação mais ricos via Open WebUI.

Q13 — Quais informações geradas por IA precisaram ser verificadas, corrigidas ou rejeitadas?

Neste caso específico, o ponto mais importante de verificação foi identificado durante a própria elaboração da ficha técnica.

- **Confusão de família de modelo:** a primeira tentativa de buscar a ficha técnica partiu do nome informado pela equipe ("gemma 3n-e2b"), mas o link fornecido posteriormente apontava, na verdade, para **Gemma 4** ([gemma-4-E2B-it-qat-q4_0-gguf](#)), uma família diferente do Gemma 3n. Isso só foi percebido ao acessar diretamente a página do modelo e comparar o nome do repositório com o que havia sido documentado antes — uma resposta de IA que apenas assumisse "é o mesmo modelo, só que mais novo" teria introduzido um erro grave no relatório e potencialmente um problema de exclusividade de modelo-base no registro do Google Classroom (Seção 5.1).
- **Licença:** era necessário confirmar a licença exata deste repositório específico. O modelo Gemma 3n usa "Gemma Terms of Use" (licença própria, gated), mas este repositório de Gemma 4 declara **Apache 2.0** — uma diferença que não podia ser assumida por analogia com a família anterior e precisou ser conferida diretamente na página do Hugging Face.
- **Cadeia de derivação do checkpoint:** a relação entre os quatro repositórios (base → instruct → QAT unquantized → GGUF) não é óbvia a partir do nome do arquivo; foi necessário consultar a seção "Model tree" da própria página para confirmar que a quantização Q4_0 vem de um processo de QAT, e não de uma quantização pós-treino aplicada livremente por terceiros.
- **Tamanho do arquivo e requisitos de hardware:** o valor de 3,35 GB veio diretamente da página oficial, mas a ficha deixa claro que isso ainda precisa ser confirmado após o download real no ambiente da equipe ([du -sh](#)), já que o enunciado exige medições próprias e não apenas números citados de terceiros — e não há uma tabela oficial de RAM/VRAM mínima nesta página, então a recomendação de "8 GB de RAM" foi tratada como estimativa a validar, não como fato do model card.
- **Suporte ao português:** o card afirma treino em 140+ idiomas e suporte "out-of-the-box" a 35+ idiomas, mas sem listar quais — essa alegação não pode ser tomada como garantia de qualidade em português e precisa ser testada empiricamente pela equipe durante os experimentos.

Esta seção documenta a observação dos processos e threads associados à execução local do Ollama e do Open WebUI, com ênfase na identificação do processo principal e de processos auxiliares, PID/PPID, estados, quantidade de threads, uso de CPU e memória e relação de parentesco entre os componentes durante a inferência.

```
fagner@FagnerJunqueira:~$ echo "=== SISTEMA ==="
cat /etc/os-release | grep -E 'PRETTY_NAME|VERSION='

echo
echo "=== KERNEL ==="
uname -a

echo
echo "=== CPU ==="
nproc
lscpu | grep -E '^CPU(s\)|Model name|Thread|Core|Socket'

echo
echo "=== MEMORIA ==="
free -h

echo
echo "=== DISCO ==="
df -h /
=== SISTEMA ===
PRETTY_NAME="Ubuntu 26.04 LTS"
VERSION="26.04 (Resolute Raccoon)"

=== KERNEL ===
Linux FagnerJunqueira 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026 x86_64 GNU/Linux

=== CPU ===
12
CPU(s): 12
Model name: 13th Gen Intel(R) Core(TM) i5-13420H
Thread(s) per core: 2
Core(s) per socket: 6
Socket(s): 1

=== MEMORIA ===
          total        used        free      shared  buff/cache   available
Mem:      3.6Gi        463Mi        2.7Gi        3.5Mi        598Mi        3.2Gi
Swap:      1.0Gi           0B        1.0Gi

=== DISCO ===
Filesystem      Size  Used Avail Use% Mounted on
/dev/sdd        1007G  1.3G  955G   1% /
```

O serviço Ollama foi executado como unidade do systemd e permaneceu ativo durante os testes. O Open WebUI foi executado em contêiner Docker e utilizado como interface para iniciar as requisições ao modelo.

Metodologia de observação

A coleta foi realizada com a aplicação em funcionamento e com o modelo em inferência. Foram utilizados os comandos recomendados no roteiro da atividade, complementados por filtros para facilitar a identificação dos processos relevantes.

```
pgrep -af ollama
```

```
ps -eo pid,ppid,stat,ni,pri,psr,pcpu,pmem,nlwp,comm --sort=-pcpu | head -20
```

```
ps -p <PID> -o pid,ppid,stat,ni,pri,psr,pcpu,pmem,nlwp,cmd
```

```
pstree -p <PID>
```

```
ps -L -p <PID> -o pid,ppid,lwp,nlwp,psr,stat,pcpu,comm
```

top -H -p <PID>

sudo docker ps

sudo docker top open-webui

```
fagner@FagnerJunqueira:~$ pgrep -af ollama
1750 /usr/local/bin/ollama serve
fagner@FagnerJunqueira:~$ sudo docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
c81550f6db12   ghcr.io/open-webui/open-webui:main "bash start.sh"         4 minutes ago   Up 4 minutes   0.0.0.0:3000->8080/tcp, [::]:3000->8080/tcp   open-webui
fagner@FagnerJunqueira:~$ sudo docker top open-webui
UID            PID            PPID           C             STIME         TTY           TIME          CMD
root           1877           1854           46            18:11         ?             00:02:11      /usr/local/bin/python3 -m uvicorn open_webui.main:app --host 0.0.0.0
--port 8080 --forwarded-allow-ips * --workers 1 --ws-per-message-deflate true
```

Resultados

Processo principal do Ollama

O processo principal do runtime foi identificado como /usr/local/bin/ollama serve, PID 1750. Seu PPID foi 1, correspondente ao systemd. Nas capturas realizadas, o processo permaneceu com estado Ssl e apresentou 27 threads. O consumo observado ficou aproximadamente entre 10,6% e 14,9% de CPU e entre 0,4% e 1,2% de memória.

```
fagner@FagnerJunqueira:~$ ps -p 1750 -o pid,ppid,stat,ni,pri,psr,pcpu,pmem,nlwp,cmd
  PID   PPID  STAT  NI PRI PSR %CPU %MEM NLWP CMD
  1750     1  Ssl   0  19   4 16.6  0.5   27 /usr/local/bin/ollama serve
fagner@FagnerJunqueira:~$ pstree -p 1750
ollama(1750)
├── {ollama}(1755)
├── {ollama}(1756)
├── {ollama}(1757)
├── {ollama}(1758)
├── {ollama}(1759)
├── {ollama}(1760)
├── {ollama}(1761)
├── {ollama}(1762)
├── {ollama}(1763)
├── {ollama}(1764)
├── {ollama}(1770)
├── {ollama}(1771)
├── {ollama}(1963)
├── {ollama}(1964)
├── {ollama}(1965)
├── {ollama}(1966)
├── {ollama}(1967)
├── {ollama}(1968)
├── {ollama}(1969)
├── {ollama}(1970)
├── {ollama}(1971)
├── {ollama}(1972)
├── {ollama}(1973)
├── {ollama}(1974)
├── {ollama}(1985)
└── {ollama}(1986)
```

```
fagner@FagnerJunqueira:~$ ps -L -p 1750 -o pid,ppid,lwp,nlwp,psr,stat,pcpu,comm
  PID    PPID      LWP  NLWP  PSR  STAT  %CPU  COMMAND
  1750      1    1750    27    4  Ssl   0.0  ollama
  1750      1    1755    27   10  Ssl   1.4  ollama
  1750      1    1756    27    6  Ssl   0.0  ollama
  1750      1    1757    27    1  Ssl   0.3  ollama
  1750      1    1758    27    0  Ssl   0.6  ollama
  1750      1    1759    27   11  Ssl   0.0  ollama
  1750      1    1760    27    8  Ssl   1.2  ollama
  1750      1    1761    27    8  Ssl   0.4  ollama
  1750      1    1762    27    7  Ssl   0.3  ollama
  1750      1    1763    27    7  Ssl   0.2  ollama
  1750      1    1764    27    0  Ssl   0.6  ollama
  1750      1    1770    27    6  Ssl   0.3  ollama
  1750      1    1771    27    2  Ssl   1.0  ollama
  1750      1    1963    27    5  Ssl   2.5  ollama
  1750      1    1964    27    9  Ssl   0.4  ollama
  1750      1    1965    27    8  Ssl   0.5  ollama
  1750      1    1966    27    8  Ssl   1.0  ollama
  1750      1    1967    27   10  Ssl   0.4  ollama
  1750      1    1968    27    1  Ssl   0.4  ollama
  1750      1    1969    27    6  Ssl   0.5  ollama
  1750      1    1970    27    5  Ssl   0.3  ollama
  1750      1    1971    27   10  Ssl   1.0  ollama
  1750      1    1972    27    0  Ssl   1.7  ollama
  1750      1    1973    27    8  Ssl   0.5  ollama
  1750      1    1974    27    3  Ssl   0.0  ollama
  1750      1    1985    27    5  Ssl   0.2  ollama
  1750      1    1986    27    2  Ssl   0.4  ollama
```

Processo auxiliar de inferência

Durante a inferência foi identificado um processo auxiliar denominado llama-server. Em uma das execuções ele apareceu com PID 2790 e PPID 1750, comprovando que era filho direto do processo ollama serve. O processo concentrou a maior parte da carga computacional associada à geração da resposta.

```
fagner@FagnerJunqueira:~$ pgrep -af ollama
1750 /usr/local/bin/ollama serve
2790 /usr/local/lib/ollama/llama-server --model /usr/share/ollama/.ollama/models/blobs/sha256-fa01b55b07ee70a54c6dae3903c783afe658664312529ea57175cb5f8dec6634 --port 40745 --host 127.0.0.1 --no-webui --offline -c 131072 -np 1 --log-verbosity 4 --no-log-prefix --no-log-timestamps --mmapro /usr/share/ollama/.ollama/models/blobs/sha256-021059cce659fe7f9170d5599761d7bbaf644b798dab9503aca30dc43e6beb14 --flash-attn auto -b 512 -ub 512 --context-shift --keep 4
fagner@FagnerJunqueira:~$ ps -eo pid,ppid,stat,ni,pri,psr,pcpu,pmem,nlwp,comm --sort=-pcpu | head -20
  PID    PPID  STAT  NI  PRI  PSR  %CPU  PMEM  NLWP  COMMAND
  2859    352  R+    0   19    0  100  0.1    1  ps
  2860    352  R+    0   19    0  100  0.0    1  head
  2790    1750  RL    0   19    6  97.6  49.9   18  llama-server
  1750      1  Ssl    0   19    4  19.6  0.5   27  ollama
   46      1  Ss+   -1   20    0    3.0  0.1    1  systemd-journal
   84      1  Ss    0   19    0    0.6  0.1    1  systemd-udevd
  1217   1216  Ssl+   0   19   11    8.2  0.3   17  docker-desktop-
    1      0  Ss    0   19    7    0.2  0.1    1  systemd
   170      1  Ssl    0   19    7    0.1  0.0    4  rsyslogd
  1403      1  Ssl    0   19   10    0.0  0.0    4  polkitd
  1013      1  Ss    0   19    9    0.0  0.0    1  systemd
   257   104  Ss    0   19    4    0.0  0.0    1  chronyd
   398      1  Ss    0   19   11    0.0  0.0    1  systemd
   352   351  Ss    0   19    2    0.0  0.0    1  bash
   106      1  Ss    0   19    4    0.0  0.0    1  dbus-daemon
   149      1  Ss    0   19    0    0.0  0.0    1  systemd-logind
   105      1  Ss    0   19    1    0.0  0.0    1  cron
    78      1  Ss    0   19    3    0.0  0.0    1  systemd-resolve
   351   350  Ss    0   19    4    0.0  0.0    1  Relay(352)
```

```
fagner@FagnerJunqueira:~$ ps -p 2790 -o pid,ppid,stat,ni,pri,psr,pcpu,pmem,nlwp,cwd
  PID    PPID  STAT  NI  PRI  PSR  %CPU  PMEM  NLWP  CWD
  2790    1750  RL    0   19    6  250  20.1   24  /usr/local/lib/ollama/llama-server --model /usr/share/ollama/.ollama/models/blobs/sha256-fa01b55b07ee70a54c6dae3903c783afe658664312529ea57175cb5f8dec6634 --port
```

Nas diferentes capturas, o llama-server variou de 18 a 24 threads. O uso de CPU também variou conforme o instante da medição, chegando a aproximadamente 250% em uma captura. Em Linux, valores acima de 100% são possíveis em processos multithread porque o percentual pode representar o uso agregado de mais de um processador lógico. O uso de memória variou entre as capturas, chegando a 49,9% em um dos momentos de maior carga.

Relação pai-filho e árvore de execução

O comando `ps tree` evidenciou a hierarquia entre os processos. O processo `ollama(1750)` possuía como filho direto o `llama-server(2790)`. Além disso, tanto o `Ollama` quanto o `llama-server` exibiram diversas threads internas, representadas entre chaves na saída do `ps tree`.

Hierarquia observada:

`systemd (PID 1)`

- └─ `ollama serve (PID 1750)`
 - └─ `llama-server (PID 2790)`
 - └─ múltiplas threads de inferência

```
fagner@FagnerJunqueira:~$ ps tree -p 1750
ollama(1750)─llama-server(2790)─{llama-server}(2792)
                                     {llama-server}(2793)
                                     {llama-server}(2794)
                                     {llama-server}(2796)
                                     {llama-server}(2797)
                                     {llama-server}(2798)
                                     {llama-server}(2799)
                                     {llama-server}(2800)
                                     {llama-server}(2801)
                                     {llama-server}(2802)
                                     {llama-server}(2803)
                                     {llama-server}(2804)
                                     {llama-server}(2805)
                                     {llama-server}(2806)
                                     {llama-server}(2807)
                                     {llama-server}(2808)
                                     {llama-server}(2809)
                                     {llama-server}(2865)
                                     {llama-server}(2866)
                                     {llama-server}(2867)
                                     {llama-server}(2868)
                                     {llama-server}(2869)
                                     {llama-server}(2870)
                                     {ollama}(1755)
                                     {ollama}(1756)
                                     {ollama}(1757)
                                     {ollama}(1758)
                                     {ollama}(1759)
                                     {ollama}(1760)
                                     {ollama}(1761)
                                     {ollama}(1762)
                                     {ollama}(1763)
                                     {ollama}(1764)
                                     {ollama}(1770)
                                     {ollama}(1771)
                                     {ollama}(1963)
                                     {ollama}(1964)
                                     {ollama}(1965)
                                     {ollama}(1966)
                                     {ollama}(1967)
                                     {ollama}(1968)
                                     {ollama}(1969)
                                     {ollama}(1970)
                                     {ollama}(1971)
                                     {ollama}(1972)
                                     {ollama}(1973)
                                     {ollama}(1974)
                                     {ollama}(1985)
                                     {ollama}(1986)
```

Threads observadas

A listagem detalhada com `ps -L` mostrou que as threads do llama-server compartilhavam o mesmo PID do processo, porém possuíam valores LWP distintos. Isso evidencia a implementação multithread da inferência. Durante uma captura foram observadas 24 threads.

Entre as threads também apareceram identificadores associados à execução CUDA, como `cuda-EvtHandlr` e uma thread com nome iniciado por `cuda`, o que é coerente com a detecção de GPU pelo Ollama. Algumas threads apresentaram uso significativo de CPU simultaneamente, demonstrando paralelismo durante a execução.

```
fagner@FagnerJunqueira:~$ ps -L -p 2790 -o pid,ppid,lwp,nlwp,psr,stat,pcpu,comm
  PID   PPID    LWP  NLWP  PSR  STAT  %CPU  COMMAND
  2790   1750   2790    24    3  Sl    76.0  llama-server
  2790   1750   2792    24    8  Sl     0.0  llama-server
  2790   1750   2793    24    1  Sl     0.0  llama-server
  2790   1750   2794    24    7  Sl     0.0  cuda00001800007
  2790   1750   2796    24    4  Sl     0.0  cuda-EvtHandlr
  2790   1750   2797    24   10  Sl     0.0  llama-server
  2790   1750   2798    24   10  Sl     0.0  llama-server
  2790   1750   2799    24    2  Sl     0.1  llama-server
  2790   1750   2800    24    2  Sl     0.0  llama-server
  2790   1750   2801    24    6  Sl     0.9  llama-server
  2790   1750   2802    24   10  Sl     0.0  llama-server
  2790   1750   2803    24    5  Sl     0.0  llama-server
  2790   1750   2804    24    4  Sl     0.4  llama-server
  2790   1750   2805    24    0  Sl     0.0  llama-server
  2790   1750   2806    24    2  Sl     0.0  llama-server
  2790   1750   2807    24   10  Sl     0.1  llama-server
  2790   1750   2808    24   11  Sl     0.0  llama-server
  2790   1750   2809    24    8  Sl     0.0  llama-server
  2790   1750   2865    24    1  Sl    35.0  llama-server
  2790   1750   2866    24   11  Sl    34.3  llama-server
  2790   1750   2867    24    5  Sl    34.7  llama-server
  2790   1750   2868    24    9  Sl    34.9  llama-server
  2790   1750   2869    24    2  Sl    34.1  llama-server
  2790   1750   2870    24    4  Sl     0.2  llama-server
```

Observação com `top -H`

O comando `top -H -p 2790` permitiu visualizar as threads como entidades individuais. Em uma das capturas foram registradas 24 threads associadas ao llama-server. A maioria estava em estado de espera no instante exato da captura, enquanto algumas threads já haviam acumulado tempo expressivo de CPU, refletindo o trabalho realizado durante a inferência.

```
top - 15:28:22 up 32 min,  2 users,  load average: 2.16, 2.23, 3.16
Threads: 24 total,  6 running, 18 sleeping,  0 stopped,  0 zombie
%Cpu(s): 29.0 us,  2.4 sy,  0.0 ni, 67.0 id,  0.1 wa,  0.0 hi,  1.6 si,  0.0 st
MiB Mem : 3730.6 total,  50.0 free, 1874.7 used, 2129.2 buff/cache
MiB Swap: 1024.0 total,  31.1 free,  992.9 used. 1855.9 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
2790	ollama	20	0	57.4g	925672	360088	R	98.3	24.2	2:51.59	llama-server
2865	ollama	20	0	57.4g	925672	360088	R	52.8	24.2	1:05.07	llama-server
2868	ollama	20	0	57.4g	925672	360088	R	52.8	24.2	1:05.06	llama-server
2866	ollama	20	0	57.4g	925672	360088	R	52.5	24.2	1:03.67	llama-server
2867	ollama	20	0	57.4g	925672	360088	R	52.5	24.2	1:04.63	llama-server
2869	ollama	20	0	57.4g	925672	360088	R	52.5	24.2	1:03.64	llama-server
2804	ollama	20	0	57.4g	925672	360088	S	1.0	24.2	0:01.12	llama-server
2870	ollama	20	0	57.4g	925672	360088	S	0.3	24.2	0:00.41	llama-server
2792	ollama	20	0	57.4g	925672	360088	S	0.0	24.2	0:00.01	llama-server
2793	ollama	20	0	57.4g	925672	360088	S	0.0	24.2	0:00.00	llama-server
2794	ollama	20	0	57.4g	925672	360088	S	0.0	24.2	0:00.00	cuda00001800007
2796	ollama	20	0	57.4g	925672	360088	S	0.0	24.2	0:00.15	cuda-EvtHandlr
2797	ollama	20	0	57.4g	925672	360088	S	0.0	24.2	0:00.11	llama-server
2798	ollama	20	0	57.4g	925672	360088	S	0.0	24.2	0:00.03	llama-server
2799	ollama	20	0	57.4g	925672	360088	S	0.0	24.2	0:00.86	llama-server
2800	ollama	20	0	57.4g	925672	360088	S	0.0	24.2	0:00.00	llama-server
2801	ollama	20	0	57.4g	925672	360088	S	0.0	24.2	0:01.91	llama-server
2802	ollama	20	0	57.4g	925672	360088	S	0.0	24.2	0:00.00	llama-server
2803	ollama	20	0	57.4g	925672	360088	S	0.0	24.2	0:00.00	llama-server
2805	ollama	20	0	57.4g	925672	360088	S	0.0	24.2	0:00.01	llama-server
2806	ollama	20	0	57.4g	925672	360088	S	0.0	24.2	0:00.00	llama-server
2807	ollama	20	0	57.4g	925672	360088	S	0.0	24.2	0:00.24	llama-server
2808	ollama	20	0	57.4g	925672	360088	S	0.0	24.2	0:00.01	llama-server
2809	ollama	20	0	57.4g	925672	360088	S	0.0	24.2	0:00.13	llama-server

```
top - 15:27:37 up 31 min,  2 users,  load average: 0.87, 2.05, 3.15
Threads: 24 total,  0 running, 24 sleeping,  0 stopped,  0 zombie
%Cpu(s):  0.7 us,  0.7 sy,  0.0 ni, 98.6 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem : 3730.6 total,  32.4 free, 1849.3 used, 2172.5 buff/cache
MiB Swap: 1024.0 total,  54.1 free,  969.9 used. 1881.4 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
2790	ollama	20	0	57.3g	847816	348776	S	0.0	22.2	2:31.25	llama-server
2792	ollama	20	0	57.3g	847816	348776	S	0.0	22.2	0:00.01	llama-server
2793	ollama	20	0	57.3g	847816	348776	S	0.0	22.2	0:00.00	llama-server
2794	ollama	20	0	57.3g	847816	348776	S	0.0	22.2	0:00.00	cuda00001800007
2796	ollama	20	0	57.3g	847816	348776	S	0.0	22.2	0:00.12	cuda-EvtHandlr
2797	ollama	20	0	57.3g	847816	348776	S	0.0	22.2	0:00.08	llama-server
2798	ollama	20	0	57.3g	847816	348776	S	0.0	22.2	0:00.03	llama-server
2799	ollama	20	0	57.3g	847816	348776	S	0.0	22.2	0:00.33	llama-server
2800	ollama	20	0	57.3g	847816	348776	S	0.0	22.2	0:00.00	llama-server
2801	ollama	20	0	57.3g	847816	348776	S	0.0	22.2	0:01.79	llama-server
2802	ollama	20	0	57.3g	847816	348776	S	0.0	22.2	0:00.00	llama-server
2803	ollama	20	0	57.3g	847816	348776	S	0.0	22.2	0:00.00	llama-server
2804	ollama	20	0	57.3g	847816	348776	S	0.0	22.2	0:00.96	llama-server
2805	ollama	20	0	57.3g	847816	348776	S	0.0	22.2	0:00.01	llama-server
2806	ollama	20	0	57.3g	847816	348776	S	0.0	22.2	0:00.00	llama-server
2807	ollama	20	0	57.3g	847816	348776	S	0.0	22.2	0:00.24	llama-server
2808	ollama	20	0	57.3g	847816	348776	S	0.0	22.2	0:00.00	llama-server
2809	ollama	20	0	57.3g	847816	348776	S	0.0	22.2	0:00.00	llama-server
2865	ollama	20	0	57.3g	847816	348776	S	0.0	22.2	0:56.91	llama-server
2866	ollama	20	0	57.3g	847816	348776	S	0.0	22.2	0:55.91	llama-server
2867	ollama	20	0	57.3g	847816	348776	S	0.0	22.2	0:56.53	llama-server
2868	ollama	20	0	57.3g	847816	348776	S	0.0	22.2	0:56.87	llama-server
2869	ollama	20	0	57.3g	847816	348776	S	0.0	22.2	0:55.48	llama-server
2870	ollama	20	0	57.3g	847816	348776	S	0.0	22.2	0:00.38	llama-server

```
fagner@fagnerjunqueira:~$ ps -p 2790 -o pid,ppid,stat,ni,pri,psr,pcpu,pmem,nlwp,cmd
PID  PPID STAT NI PRI PSR %CPU %MEM NLWP CMD
2790 1750 S1 0 19 4 150 24.2 24 /usr/local/lib/ollama/llama-server --model /usr/share/ollama/.ollama/models/blobs/sha256-fa401b55b07ee70a54c6dae3903c783a6e65864312529ea57175cb5f8dec6634 --port
```

Respostas às questões de análise

Q4 — Quais processos e threads foram observados durante a inicialização e a inferência?

Durante a execução foram observados principalmente o processo ollama serve, responsável pelo serviço e coordenação do runtime, e o processo auxiliar llama-server, utilizado na execução do modelo. O ollama serve foi identificado com PID 1750, PPID 1, estado Ssl e 27 threads. Durante a inferência, o llama-server apareceu como filho do Ollama, com PID 2790 e PPID 1750. Dependendo do instante da medição, apresentou entre 18 e 24 threads, estados Rl ou Sl e elevado consumo de CPU e memória. Em uma captura apresentou 97,6% de CPU e 49,9% de memória; em outra, o uso agregado de CPU chegou a aproximadamente 250%. O ps -L e o top -H confirmaram a execução multithread, inclusive com threads relacionadas à CUDA.

Q5 — Qual é a relação de parentesco e responsabilidade entre os processos identificados?

O processo ollama serve possui PPID 1 e é gerenciado pelo systemd, funcionando como processo principal do runtime. O llama-server possui PPID 1750, logo é filho direto do processo Ollama. A árvore obtida com pstree confirmou a hierarquia systemd -> ollama -> llama-server. O ollama serve é responsável por disponibilizar o serviço, receber e coordenar requisições e gerenciar o ciclo de execução do modelo. O llama-server concentra a execução computacional da inferência e utiliza várias threads para distribuir o trabalho entre processadores lógicos e recursos de GPU. O Open WebUI, por sua vez, é executado em um contêiner Docker independente; ele não é filho do Ollama, mas atua como camada de aplicação que envia requisições ao runtime por HTTP.

Discussão

As medições mostram uma separação clara de responsabilidades. O processo ollama serve permanece como serviço relativamente estável, enquanto o llama-server concentra a carga variável da inferência. A alteração no número de threads e no consumo de recursos entre diferentes capturas demonstra que a execução não é estática: o runtime adapta sua atividade conforme o estado do modelo e da requisição. A utilização de múltiplas threads e a presença de threads CUDA reforçam que a inferência explora paralelismo e aceleração por GPU.

Também é importante observar que uma captura de ps, top ou htop representa apenas um instante. Por isso, valores de estado, CPU, memória e quantidade de threads podem variar entre capturas. A análise foi baseada no conjunto das evidências, e não em uma única medição isolada.

Conclusão

A observação dos processos e threads permitiu relacionar conceitos de Sistemas Operacionais a uma aplicação real de IA generativa. Foi possível identificar a hierarquia de processos, distinguir o serviço principal do processo de inferência, observar execução multithread, acompanhar estados e consumo de recursos e verificar a separação arquitetural entre a interface Open WebUI e o runtime Ollama. Os resultados fornecem evidências suficientes para responder às questões Q4 e Q5 da atividade.

6 - Katyane dos Santos

Realizar e documentar os experimentos correspondentes às configurações **C1** e **C2**, avaliando o comportamento do modelo **google/gemma-4-E2B-it-qat-q4_0-gguf** durante a execução local com Ollama.

A configuração **C1** representa o cenário de referência (*baseline*), no qual apenas uma requisição é executada por vez. A configuração **C2** mantém o mesmo modelo, prompts e ambiente experimental, porém realiza **três requisições simultâneas**, permitindo analisar o impacto da concorrência sobre o tempo de resposta e o consumo de recursos computacionais.

Também fizeram parte da tarefa a coleta e comparação das métricas de **TTFT (Time to First Token)**, tempo total de execução, quantidade de tokens gerados, taxa de geração em tokens por segundo, além do monitoramento complementar de **CPU, memória RAM e número de threads**.

6.1 Ambiente experimental

Os experimentos foram realizados em um computador com **Windows 11**, utilizando o **WSL2** como ambiente Linux.

A configuração utilizada foi:

- Processador: Intel Core i5-12450H;
- 8 núcleos e 12 processadores lógicos no equipamento;
- Limite configurado para o WSL2: **8 processadores lógicos**;
- Memória destinada ao WSL2: **5 GB de RAM**;
- Swap configurada: **4 GB**;
- Sistema no WSL2: Ubuntu 26.04.1 LTS;
- Ollama: versão 0.34.0;
- Open WebUI executado através do Docker Desktop;
- Modelo: [google/gemma-4-E2B-it-qat-q4_0-gguf](#);
- Modelo no Ollama: [hf.co/google/gemma-4-E2B-it-qat-q4_0-gguf:latest](https://ollama.com/library/google/gemma-4-E2B-it-qat-q4_0-gguf);

- Quantização: Q4_0;
- Tamanho aproximado do modelo: 4,3 GB;
- Contexto utilizado: 4096 tokens;
- Execução do modelo realizada em CPU.

O Open WebUI foi configurado para acessar o Ollama através da porta local [11434](#). Durante os ajustes da interface, o modo de raciocínio foi desativado e a chamada de função foi configurada como **Legado**, de forma a evitar que o conteúdo enviado ultrapassasse o limite de contexto disponível.

6.2 Metodologia dos experimentos

Para manter a comparação entre C1 e C2, foram utilizados o mesmo modelo, ambiente, quantização e prompts. A principal variável modificada foi o número de requisições executadas simultaneamente.

Foram utilizadas duas cargas de trabalho:

Carga pequena

Prompt:

Explique em 3 frases o que é um sistema operacional.

A entrada apresentou **22 tokens de prompt**.

Carga grande

Prompt:

Explique detalhadamente como um sistema operacional gerencia os recursos de um computador. Na resposta, aborde os seguintes aspectos: gerenciamento de processos, escalonamento da CPU, threads, memória principal, memória virtual, sistema de arquivos, dispositivos de entrada e saída, interrupções, chamadas de sistema e mecanismos de proteção e segurança. Explique também como esses componentes se relacionam durante a execução simultânea de vários programas. Organize a explicação de maneira didática, utilizando aproximadamente 10 parágrafos.

A entrada apresentou **120 tokens de prompt**.

Para cada configuração foram realizadas **duas repetições com a carga pequena e duas repetições com a carga grande**.

Antes de cada repetição oficial, o modelo era descarregado utilizando o comando:

```
ollama stop hf.co/google/gemma-4-E2B-it-qat-q4_0-gguf:latest
```

Esse procedimento foi utilizado para que cada repetição começasse com o modelo fora da memória, incluindo o custo de carregamento inicial.

As requisições foram enviadas diretamente para a API local do Ollama:

```
http://127.0.0.1:11434/api/generate
```

Foi utilizado `stream=true`, permitindo medir o momento em que o primeiro fragmento de resposta era recebido.

O **TTFT** foi calculado como o intervalo entre o envio da requisição e o recebimento do primeiro conteúdo gerado pelo modelo.

A taxa de geração foi calculada a partir de:

$$\text{tokens/s} = \text{eval_count} / (\text{eval_duration} / 1.000.000.000)$$

6.3 Configuração C1 — Baseline

Na configuração C1 foi executada **uma requisição por vez**, representando o cenário de referência para comparação com a execução concorrente.

C1 — carga pequena

Repetição	TTFT	Tempo total	Tokens de saída	Tokens/s
1	84,667 s	103,863 s	81	6,573
2	73,574 s	78,705 s	76	14,959

O TTFT médio da carga pequena foi de aproximadamente:

79,12 segundos.

C1 — carga grande

Repetição	TTFT	Tempo total	Tokens de saída	Tokens/s
1	92,794 s	195,142 s	1518	14,991
2	86,351 s	163,459 s	1061	13,952

O TTFT médio da carga grande foi:

89,57 segundos.

Os resultados mostram que a carga grande exigiu maior tempo total de execução devido principalmente à maior quantidade de tokens gerados.

6.4 Configuração C2 — Três requisições simultâneas

Na configuração C2 foram enviadas **três requisições ao mesmo tempo**.

Foi utilizado um script Python com:

```
ThreadPoolExecutor(max_workers=3)
```

Cada repetição da C2 corresponde, portanto, a um lote contendo três solicitações concorrentes.

C2 — carga pequena

Repetição 1

Requisição	TTFT	Tempo total	Tokens de saída	Tokens/s
1	73,953 s	78,216 s	85	16,383
2	81,786 s	85,847 s	74	18,387

3	78,362 s	81,642 s	60	18,641
---	----------	----------	----	--------

Tempo total do lote: **85,917 s**.

Total gerado: **219 tokens**.

Repetição 2

Requisição	TTFT	Tempo total	Tokens de saída	Tokens/s
1	71,605 s	76,311 s	73	18,683
2	63,737 s	71,467 s	80	13,863
3	76,627 s	80,857 s	74	17,024

Tempo total do lote: **80,874 s**.

Total gerado: **227 tokens**.

Na carga pequena, a diferença em relação à configuração C1 foi relativamente limitada.

C2 — carga grande

Repetição 1

Requisição	TTFT	Tempo total	Tokens de saída	Tokens/s
1	232,720 s	300,166 s	1108	16,470

2	93,797 s	165,570 s	1119	15,787
3	165,753 s	232,506 s	1085	16,311

Tempo total do lote: **300,189 s.**

Total gerado: **3312 tokens.**

Repetição 2

Requisição	TTFT	Tempo total	Tokens de saída	Tokens/s
1	216,354 s	284,914 s	1134	16,594
2	157,940 s	216,186 s	972	16,753
3	83,400 s	157,752 s	1171	15,892

Tempo total do lote: **284,947 s.**

Total gerado: **3277 tokens.**

Considerando as seis requisições da carga grande, o TTFT médio da C2 foi de aproximadamente:

158,33 segundos.

6.5 Comparação entre C1 e C2

A maior diferença entre as configurações apareceu na carga grande.

Métrica	C1	C2
---------	----	----

Requisições simultâneas	1	3
TTFT médio — carga grande	89,57 s	158,33 s
CPU média — monitoramento grande	150,31%	231,00%
RAM média — monitoramento grande	3,38 GB	3,39 GB
Máximo de threads	15	15

O TTFT passou de **89,57 s para 158,33 s**, representando um aumento de aproximadamente:

76,8%.

Esse resultado indica que, no ambiente analisado, permitir três solicitações simultâneas aumentou a concorrência pelos recursos computacionais e prejudicou a latência individual das requisições na carga mais pesada.

6.6 Monitoramento de CPU, RAM e threads

Além das repetições oficiais, foram realizadas execuções adicionais destinadas especificamente ao monitoramento dos recursos utilizados pelo processo de inferência.

É importante destacar que essas medições **não correspondem exatamente às mesmas execuções utilizadas nas tabelas de TTFT**, sendo utilizadas como evidência complementar do comportamento do sistema.

Configuração	CPU média	Pico CPU	RAM média	Pico RAM	Threads médias	Máximo
--------------	--------------	-------------	--------------	----------	-------------------	--------

C1 pequena	107,32%	137%	3195,68 MB	3991,66 MB	13,37	15
C1 grande	150,31%	227%	3381,68 MB	4039,43 MB	13,93	15
C2 pequena	104,03%	141%	3107,00 MB	3956,31 MB	13,38	15
C2 grande	231,00%	310%	3387,51 MB	3938,38 MB	14,49	15

O resultado mais significativo foi observado na CPU durante a carga grande.

A média passou de aproximadamente **150,31% na C1 para 231% na C2**.

Em Linux, um processo multithread pode apresentar utilização acima de 100%, pois esse valor representa o uso agregado de múltiplos processadores lógicos. Portanto, 231% corresponde aproximadamente ao equivalente à utilização de 2,31 processadores lógicos.

Em contrapartida, a memória RAM praticamente não aumentou proporcionalmente ao número de requisições.

Isso ocorre porque as requisições compartilham os pesos do mesmo modelo carregado em memória, em vez de carregar três cópias completas do modelo.

Q7 — Qual foi o impacto da concorrência no desempenho e no uso de recursos?

A configuração C2 permitiu que três requisições fossem submetidas simultaneamente, mas isso não resultou em melhora da responsividade individual.

Na carga pequena, os resultados permaneceram relativamente próximos aos observados na C1. Entretanto, na carga grande, o TTFT médio aumentou de **89,57 segundos para 158,33 segundos**, aproximadamente **76,8% de aumento**.

Também foi observado aumento relevante no consumo de CPU durante as execuções monitoradas, passando de uma média de **150,31% na C1 grande para 231% na C2 grande**.

Assim, os experimentos demonstram um *trade-off* entre concorrência e latência. A concorrência permite manter várias solicitações em andamento, porém, em um ambiente limitado por CPU e memória, as requisições passam a disputar os mesmos recursos computacionais.

Q8 — Quais recursos do sistema operacional passaram a ser mais disputados?

Durante a execução concorrente, a principal competição ocorreu pelo tempo de processamento da **CPU**.

As requisições também disputam recursos relacionados à memória, largura de banda de memória, estados de contexto do modelo e estruturas internas utilizadas durante a inferência.

O ambiente utilizado possuía apenas **5 GB de RAM destinados ao WSL2**, enquanto o modelo ocupava aproximadamente **4,3 GB em armazenamento e cerca de 3 a 4 GB de memória durante a execução**. Dessa forma, a utilização de **swap** também se tornou relevante para permitir a execução quando a pressão de memória aumentava.

Entretanto, a RAM utilizada não triplicou ao passar de uma para três requisições porque os pesos do modelo permanecem compartilhados.

Portanto, os resultados indicam que a concorrência aumentou principalmente a pressão sobre a **CPU**, enquanto memória e threads permaneceram relativamente próximas entre C1 e C2.

Dificuldades encontradas

Durante a realização dos experimentos ocorreram algumas dificuldades relacionadas às limitações do equipamento e à configuração do ambiente.

O computador utilizado possui **8 GB de memória RAM física**, o que exigiu limitar cuidadosamente os recursos destinados ao WSL2. Foram configurados **5 GB de RAM e 4 GB de swap**.

Outro problema ocorreu inicialmente no Open WebUI, quando uma requisição excedeu o contexto máximo de **4096 tokens**. O problema foi contornado mantendo o contexto em 4096, desativando o modo de raciocínio e configurando a chamada de função como **Legado**, reduzindo a quantidade de tokens adicionais gerados pela interface.

Também foi observado que o carregamento do modelo após sua remoção da memória apresentava tempo elevado. Entretanto, esse comportamento foi mantido intencionalmente nos experimentos oficiais para que todas as repetições fossem realizadas utilizando a mesma metodologia de inicialização a frio.

Conclusão dos experimentos C1 e C2

Os experimentos permitiram comparar o comportamento do modelo Gemma nas configurações C1 e C2.

A configuração C1 apresentou melhor latência individual, especialmente na carga grande. Já a C2 possibilitou a submissão de três solicitações simultaneamente, porém apresentou maior competição pelos recursos computacionais disponíveis.

O impacto tornou-se mais evidente no cenário de maior carga, no qual o TTFT médio aumentou aproximadamente **76,8%** e o uso médio de CPU nas execuções monitoradas passou de **150,31% para 231%**.

Por outro lado, a utilização de memória permaneceu próxima entre as duas configurações, indicando o compartilhamento dos pesos do modelo entre as requisições.

Dessa forma, os resultados evidenciam a existência de um compromisso entre **concorrência, latência e utilização de recursos**. Em equipamentos com memória e capacidade de processamento limitadas, aumentar o número de solicitações simultâneas pode permitir atender múltiplos usuários, mas pode elevar consideravelmente o tempo de resposta individual.

7 - Vênisson Cardoso dos Santos

7.1 Ambiente experimental

Os experimentos da Configuração C3 foram realizados em uma máquina virtual executada no Oracle VirtualBox, utilizando Ubuntu 24.04.4 LTS como sistema convidado.

A configuração utilizada foi:

- Processador disponibilizado à VM: AMD Ryzen 7 7435HS;
- 2 processadores lógicos disponíveis para a máquina virtual;
- Memória RAM da VM: aproximadamente 3,8 GiB;

- Swap configurada: 4 GiB;
- Kernel Linux: 6.8.0-134-generic;
- Arquitetura: x86_64;
- Ollama: versão 0.34.0;
- Modelo: google/gemma-4-E2B-it-qat-q4_0-gguf;
- Modelo no Ollama: hf.co/google/gemma-4-E2B-it-qat-q4_0-gguf:latest;
- Quantização: Q4_0;
- Tamanho aproximado do modelo no Ollama: 4,3 GB;
- Execução realizada integralmente em CPU.

Durante a preparação do ambiente, a VM inicialmente não possuía memória swap. Nessas condições, a primeira tentativa de carregar o modelo provocou esgotamento de memória e o processo llama-server foi encerrado pelo mecanismo OOM Killer do Linux. Para permitir a continuidade dos experimentos, foi configurada uma área de swap de 4 GiB.

7.2 Metodologia dos experimentos

Para manter a comparação controlada na Configuração C3, foram utilizados o mesmo modelo, ambiente, quantização e prompt em todas as execuções. A principal variável modificada foi o tamanho da janela de contexto configurada no Ollama.

Foram utilizadas duas condições:

Contexto curto: num_ctx = 1024;

Contexto longo: num_ctx = 4096.

O prompt utilizado em todas as execuções foi:

Explique em uma frase o que é um sistema operacional.

O modo de raciocínio adicional foi desativado com think=false, e foi utilizado stream=false. Também foi definido keep_alive=0, fazendo com que o modelo fosse descarregado ao final de cada requisição, de forma que as execuções seguintes incluíssem novamente o custo de carregamento inicial.

Nas execuções bem-sucedidas, o prompt apresentou 21 tokens. Portanto, o experimento avaliou o efeito da alteração da janela de contexto configurada, e não de um aumento efetivo do tamanho da entrada.

Foram realizadas duas execuções bem-sucedidas para cada tamanho de contexto. Durante os testes com num_ctx=4096, ocorreu ainda uma tentativa adicional que terminou em timeout durante a inicialização do llama-server, sendo essa falha mantida como evidência experimental.

7.3 Configuração C3 — Ajuste da janela de contexto

Na Configuração C3 foi mantido o mesmo modelo e o mesmo prompt, alterando apenas a janela de contexto configurada entre 1024 e 4096 tokens. O objetivo foi observar como essa mudança afetaria o tempo de execução, a velocidade de geração e a estabilidade do modelo no ambiente virtualizado.

Resultados

Execução	Janela de contexto	Resultado	Tempo total	Tempo carregamento	Tempo de geração	Velocidade
Curto 1	1024	Sucesso	392,638 s	380,046 s	8,527 s	3,635 tok/s
Curto 2	1024	Sucesso	425,296 s	382,574 s	38,033 s	0,710 tok/s
Longo 1	4096	Sucesso	511,902 s	340,999 s	33,383 s	0,869 tok/s
Longo 2	4096	Timeout	—	—	—	—
Longo 3	4096	Sucesso	407,265 s	332,563 s	72,304 s	0,360 tok/s

A execução Longo 2 foi mantida como evidência de falha experimental, pois o llama-server não ficou disponível dentro do tempo limite do Ollama.

7.4 Comparação entre as janelas de contexto

Considerando apenas as execuções concluídas com sucesso, o tempo total médio foi de 408,967 s para a janela de contexto de 1024 tokens e de 459,583 s para a janela de 4096 tokens. Isso representa um aumento aproximado de 12,4% no tempo total médio quando utilizada a janela de contexto maior.

O tempo médio de geração também aumentou, passando de 23,280 s no contexto de 1024 tokens para 52,844 s no contexto de 4096 tokens. Ao mesmo tempo, a velocidade média de geração caiu de 2,172 para 0,615 token/s, indicando maior custo computacional e menor desempenho médio na configuração com janela de contexto maior.

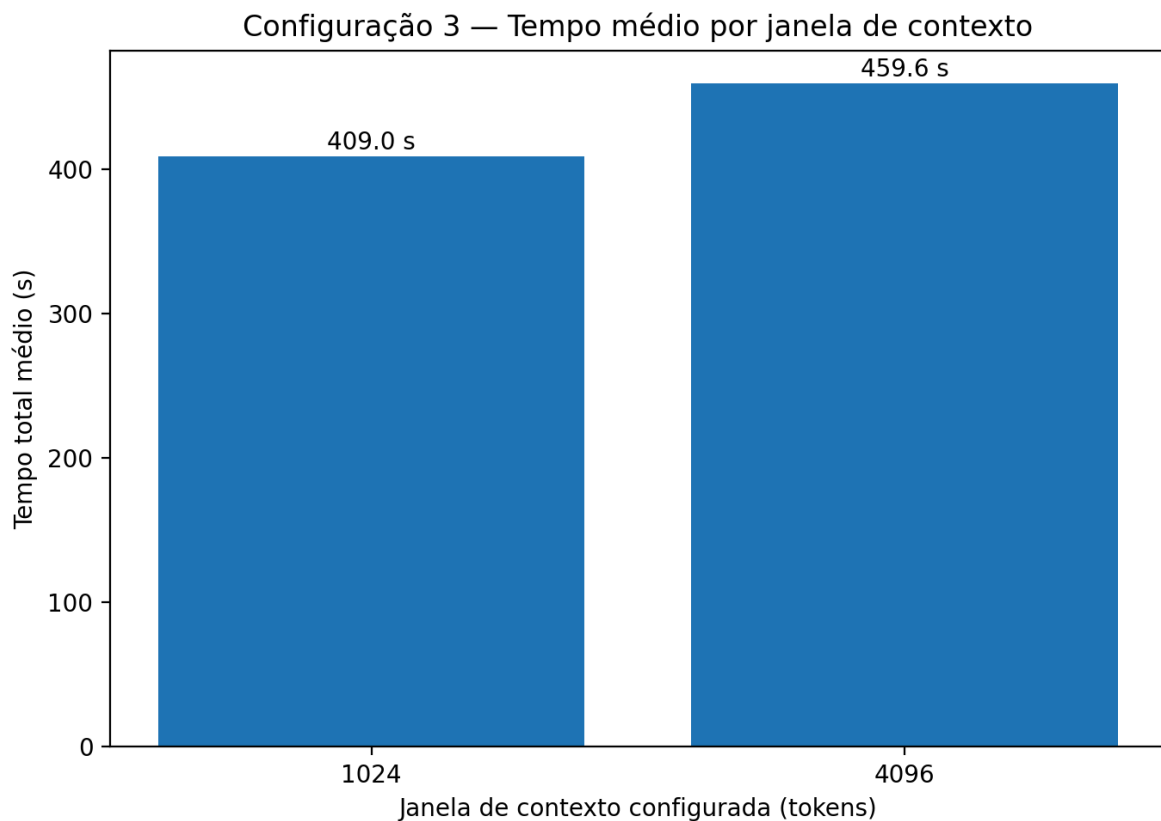


Figura — Tempo total médio por janela de contexto na Configuração C3.

7.5 Dificuldades e limitações

A principal limitação observada foi a quantidade reduzida de memória disponível na máquina virtual. Inicialmente, com aproximadamente 3,8 GiB de RAM e sem swap, o carregamento do modelo

provocou esgotamento de memória e o processo `llama-server` foi encerrado pelo mecanismo OOM Killer do Linux. Após a configuração de 4 GiB de swap, as inferências passaram a ser concluídas.

Durante os testes com janela de contexto de 4096 tokens também foi observada uma falha por timeout. Nessa execução, o llama-server permaneceu em processo de carregamento por aproximadamente dez minutos sem ficar disponível para atender à requisição. Como outras duas execuções com a mesma configuração foram concluídas com sucesso, a falha foi interpretada como evidência de instabilidade do ambiente limitado, e não como impossibilidade absoluta de utilizar a janela de 4096 tokens.

Q9 - Como entradas ou contextos maiores afetaram o desempenho e o uso de recursos?

Na Configuração C3, a janela de contexto configurada foi alterada de 1024 para 4096 tokens, mantendo-se o mesmo modelo, prompt e ambiente. Nas execuções concluídas, o tempo total médio aumentou de 408,967 s para 459,583 s, aproximadamente 12,4%. O tempo médio de geração passou de 23,280 s para 52,844 s, enquanto a velocidade média caiu de 2,172 para 0,615 token/s. Além disso, uma das três tentativas com janela de 4096 tokens terminou em timeout durante a inicialização do `llama-server`. Assim, no ambiente virtualizado utilizado, a janela de contexto maior esteve associada a maior custo de execução e menor previsibilidade.

Q10 — Como a arquitetura local contribui para privacidade, disponibilidade, autonomia e controle de dados? Quais riscos permanecem?

A arquitetura local com Ollama e Open WebUI aumenta a privacidade e a autonomia porque a inferência é realizada no próprio ambiente computacional, reduzindo a necessidade de enviar prompts e dados para serviços externos. Isso também permite maior controle sobre o modelo utilizado, as configurações de execução, o armazenamento e o acesso aos dados. Entretanto, permanecem riscos relacionados à exposição de portas e serviços locais, armazenamento de históricos e logs, permissões de acesso e limitações do próprio hardware. Nos experimentos realizados, a disponibilidade do sistema foi diretamente afetada pela quantidade de memória e capacidade de processamento da máquina virtual, demonstrando que a execução local oferece maior controle, mas também transfere ao usuário a responsabilidade pela segurança, manutenção e dimensionamento dos recursos.

7.6 Conclusão da Configuração C3

Os experimentos da Configuração C3 mostraram que a alteração da janela de contexto influenciou o desempenho e a estabilidade da execução local do modelo. A configuração de 4096 tokens apresentou maior tempo total médio, maior tempo de geração e menor velocidade média em comparação com a janela de 1024 tokens, além de registrar uma ocorrência de timeout. Os resultados também evidenciaram a forte dependência do desempenho em relação aos recursos disponíveis na máquina virtual, especialmente memória RAM, swap e capacidade de processamento. Dessa forma, a Configuração C3 demonstrou que ajustes aparentemente simples no contexto podem produzir impactos relevantes no custo computacional e na previsibilidade da inferência local.

8 - Conclusão para o Relatório

A realização deste trabalho permitiu analisar a execução local de um modelo de linguagem não apenas como uma aplicação de inteligência artificial, mas como um sistema diretamente dependente dos mecanismos e recursos administrados pelo sistema operacional. A preparação dos ambientes com Linux, Ollama e Open WebUI possibilitou executar localmente o modelo Gemma quantizado e evidenciou a importância de recursos como CPU, memória RAM, swap e virtualização para a viabilidade e o desempenho da inferência. Os diferentes ambientes utilizados pelo grupo também mostraram que a disponibilidade e a alocação desses recursos influenciam diretamente o comportamento da aplicação.

A observação da aplicação em funcionamento permitiu relacionar conceitos estudados em Sistemas Operacionais com uma carga de trabalho real. Foi possível identificar a hierarquia entre o serviço Ollama e o processo llama-server, observar a utilização de múltiplas threads durante a inferência e acompanhar estados e consumo de recursos. Dessa forma, processos, threads, concorrência e gerenciamento de recursos deixaram de ser apenas conceitos teóricos e puderam ser observados no funcionamento de uma aplicação contemporânea de IA generativa.

Os experimentos também demonstraram que não existe uma configuração ideal independente do hardware disponível. Na comparação entre C1 e C2, o aumento da concorrência permitiu processar múltiplas solicitações simultaneamente, mas aumentou a disputa por recursos e a latência individual; na carga maior, o TTFT médio aumentou aproximadamente 76,8% e o uso médio de CPU passou de 150,31% para 231%. Na configuração C3, o aumento da janela de contexto de 1024 para 4096 tokens elevou o tempo total médio de 408,967 s para 459,583 s, reduziu a velocidade média de geração e apresentou uma ocorrência de timeout. Esses resultados evidenciam um compromisso entre concorrência, tamanho de contexto, latência, estabilidade e utilização dos recursos computacionais.

Por fim, a execução local com Ollama e Open WebUI oferece maior autonomia e controle sobre o modelo, suas configurações e os dados processados, além de reduzir a necessidade de enviar prompts para serviços externos. Entretanto, esse controle também transfere ao usuário responsabilidades relacionadas à segurança, manutenção, configuração e dimensionamento do hardware. Assim, o trabalho demonstrou que a execução local de modelos de linguagem é viável, mas seu desempenho e sua disponibilidade dependem diretamente das decisões de configuração e dos recursos administrados pelo sistema operacional.

9 - Referências:

GOOGLE. *Gemma 4n*. Google AI for Developers. Disponível em: <https://ai.google.dev/gemma/docs/gemma-4n?hl=pt-br>. Acesso em: 15 set. 2026.

GOOGLE. *gemma-4-E2B-it-qat-q4_0-gguf*. Hugging Face. Disponível em: https://huggingface.co/google/gemma-4-E2B-it-qat-q4_0-gguf. Acesso em: 15 set. 2026.

OLLAMA. *gemma4n:e2b*. Ollama Library. Disponível em: <https://ollama.com/library/gemma4n:e2b>. Acesso em: 15 set. 2026.

- **Declaração de uso de IA generativa**

IA generativa:	Finalidade:	Prompt utilizado:	Saída:
Claude	Planejamento geral da atividade a partir do enunciado	"Analise TODO esse documento, e a partir da análise, quero sua ajuda com um roadmap para dividir as atividades entre os membros de forma coerente"	Sugeriu a divisão por seções do enunciado (ambiente, camada de aplicação, modelo, processos/threads, syscalls, experimentos) e recomendou o tópicos para cada membro do time, além disso, retornou um roteiro dia a dia (instalação, observação de processos/threads/syscalls, experimentos comparativos, relatório, vídeo)
Copilot (Github)	Organização da estrutura do repositório no GitHub	"A partir do Roadmap e divisão das tarefas anteriores, me ajude na criação de toda a estrutura de pastas no GitHub, de forma que facilite o trabalho dos membros"	Criou toda a estrutura do repositório, indicou em quais arquivos cada um iria ficar, a partir da pasta contexto/ , onde cada trecho do texto deveria entrar, e os comandos git add/commit/push para subir o conteúdo para a branch main do nosso repositório.
Claude	Verificação de informação crítica	"Eu preciso da ficha técnica do modelo 4n-e2b escolhido pela equipe, use esse link: [link do gemma-4-E2B-it-qat-q4_0-gguf] e do modelo gemma-3-E2B e suas principais diferenças para determinar qual o mais indicado para a trilha escolhida pela equipe"	A IA alertou sobre a divergência de família de modelo e refez a ficha técnica com os dados corretos. Apontando o 4-E2B como o mais indicado para a atividade.
Claude	Aprofundamento em um critério específico	"Descreva o mais afundo o critério formato exigido na ficha técnica"	Explicação da cadeia de derivação do modelo (checkpoint base → instruct → QAT → conversão GGUF) e da relevância de cada etapa para os conceitos de Sistemas Operacionais (armazenamento, I/O, chamadas de sistema no carregamento)
Copilot	Apoio na escrita/organização do relatório	"Revise esse parágrafo do relatório em pdf do meu grupo, verifique erros de pontuação e duplicidade de palavras"	revisou o arquivo conforme solicitado e corrigiu erros de gramática e coesão
(IA QUE VCS USARAM)			

